Construction Heuristics and Extensions

Heuristic Optimization Techniques 192.162, VU 6 ECTS

Enrico Iurlano, Günther Raidl, and Laurenz Tomandl
heuopt@ac.tuwien.ac.at

Algorithms and Complexity Group
Institute of Logic and Computation, TU Wien

WS 2026

Version from October 6, 2026



Construction Heuristics

Construction heuristics

Common scheme:

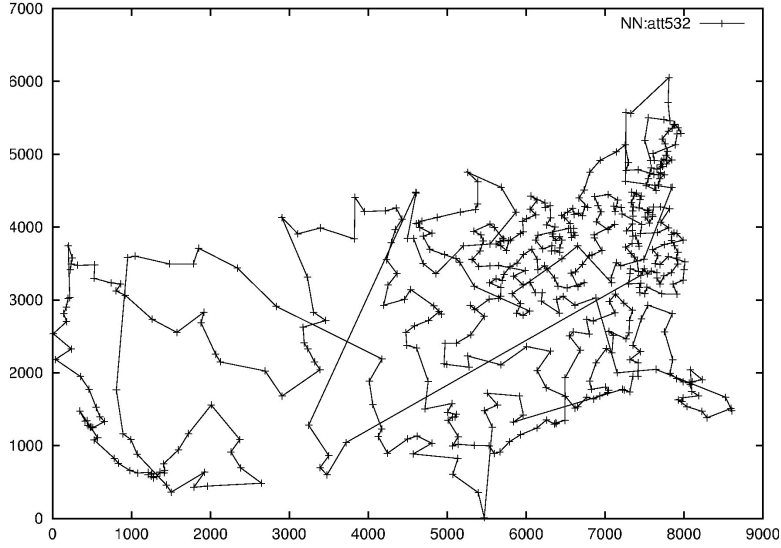
- start with an "empty" solution.
- at each step, add a new element to the solution.
- Greedy construction: use a heuristic to select the locally best element to add.
  If an element is part of the solution it is never reconsidered,
  we never "backtrack".

procedure greedy construction heuristic
begin
  $x \leftarrow$ empty solution;
  while x is not a complete solution **do**
    $e \leftarrow$ currently the most beneficial extension for x;
    $x \leftarrow x \oplus e$;
  end;
end

Examples of construction heuristics for the TSP

- Nearest neighbor heuristic:

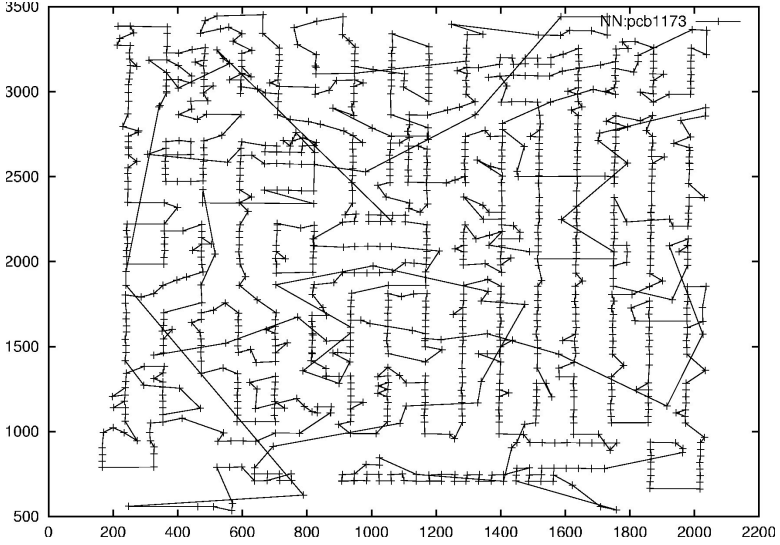
  Start at an arbitrary node and always go to a nearest unvisited node.



Examples of construction heuristics for the TSP

- Nearest neighbor heuristic:

  Start at an arbitrary node and always go to a nearest unvisited node.



Examples of construction heuristics for the TSP

- Nearest neighbor heuristic:
  Start at an arbitrary node and always go to a nearest unvisited node.
- Insertion Heuristic:
  Start with a tour of 3 nodes or the convex hull in case of Euclidean instances; insert all other node one-by-one in a cheapest possible way

  Selection strategies for the next node:
  - Random insertion: Select node at random
  - Nearest insertion: Select node with minimum distance to a node in the tour
  - Farthest insertion: Select node whose minimum distance to a node in the tour is maximum

  Insertion strategies:
  - After the nearest node in the tour
  - Inducing minimum increase of tour length

For a visualization see

https://stemlounge.com/animated-algorithms-for-the-traveling-salesman-problem/

Further construction heuristics for the TSP

- Spanning tree heuristic: approximation ratio of 2 for metric instances
- Christofides heuristic: approximation ratio of 1.5 for metric instances

Approximation ratio ρ: The algorithm returns a solution having at most ρ times the cost of an optimal solution (for minimization).

Most other heuristics may in principle return arbitrary worse solutions.

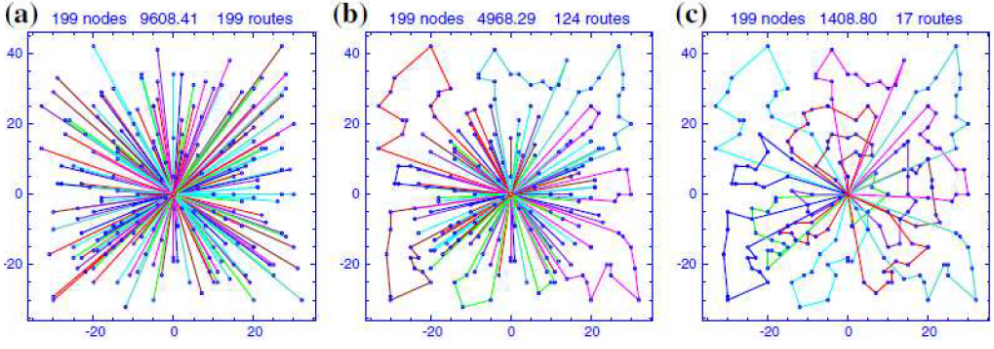
Practical performance of discussed construction heuristics

- Depends on concrete problem instance characteristics.
- Very rough average estimate for discussed TSP heuristics:
  10-20% above optimum solution cost.
- No single "always best" construction heuristic!

Savings heuristic for the (capacitated) vehicle routing problem

by Clarke and Wright (1964)

- Start with individual tours for each customer from and back to the depot.
- Consider each pair of nodes and determine the saving in the cost when connecting them directly instead of going back to the depot inbetween: $S_{ij} = c_{i0} + c_{0j} - c_{ij}$
- Consider the node pairs in non-decreasing savings order:
  Connect a node pair if both nodes are still adjacent to the depot and the resulting tour is still feasible, i.e., merging the two tours.



(from T. Anderson: Optimized vehicle routing in autonomous weed control)

Beam Search

Beam Search

- Motivation: We want to extend a one-shot solution construction heuristic towards a simple limited tree search.
- Beam Search is a width-limited breadth-first search.
- First suggested by Lowerre (1967) in the context of a speech recognition system, but the term Beam Search was coined only later by Reddy (1977).

Motivation of Beam Search

Maximum Satisfiability Problem (MAX-SAT)

Given: m clauses $C = (C_k)_{k=1,\dots,m}$ on n boolean variables $x = (x_1, x_2, \dots, x_n) \in \{0,1\}^n$

Goal: Find a variable assignment maximizing the number of satisfied clauses.

Example:

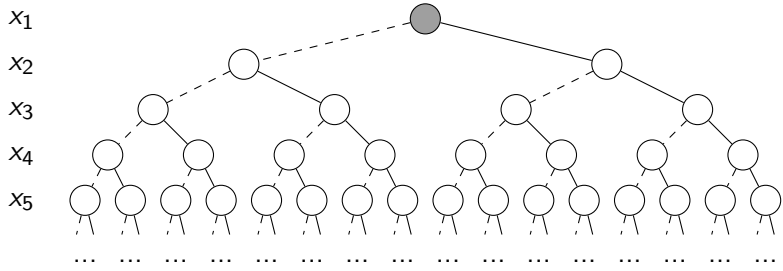
$$C_1 = x_2 \vee x_3$$
$$C_2 = x_2 \vee x_3 \vee \overline{x_4}$$
$$C_3 = \overline{x_1} \vee x_2 \vee x_4 \vee \overline{x_5}$$
$$C_4 = \overline{x_1} \vee \overline{x_2} \vee x_3$$
$$C_5 = x_1 \vee \overline{x_2} \vee \overline{x_4}$$

Here, $x^* = (1,1,1,1,1)$ satisfies all clauses and is thus an optimal solution, while, e.g., $x' = (0,0,0,0,0)$ does not satisfy C_1.

MAX-SAT is a classical NP-hard problem.

Motivation of Beam Search

Let us have a look at a naive search tree of a MAX-SAT-problem.



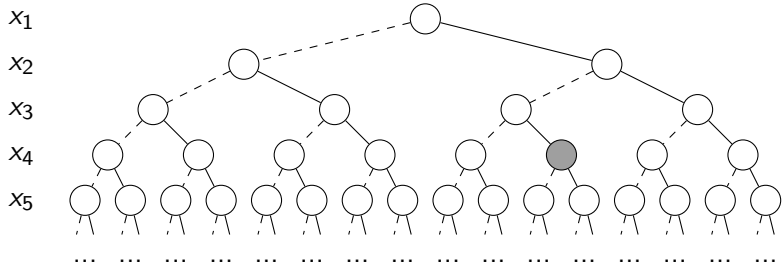
The **root node** represents the "empty" solution (no assignments done yet).

At each level i, we assign a value to x_i as follows:

- $x_i = 0$ (left branches, dashed)
- $x_i = 1$ (right branches, solid)

Motivation of Beam Search

Let us have a look at a naive search tree of a MAX-SAT-problem.

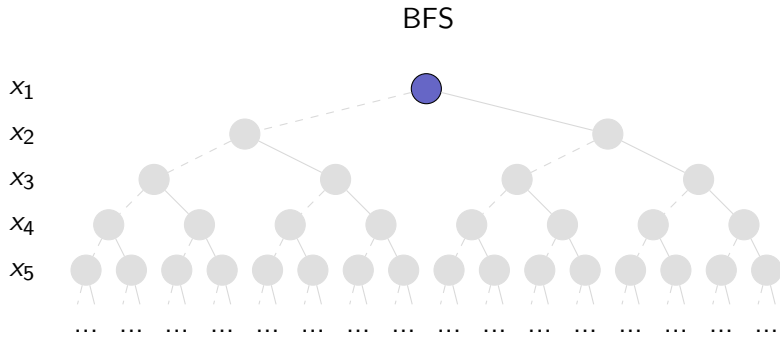


The **marked node** corresponds to the partial solution with

- $x_1 = x_3 = 1$ and
- $x_2 = 0$

Motivation of Beam Search

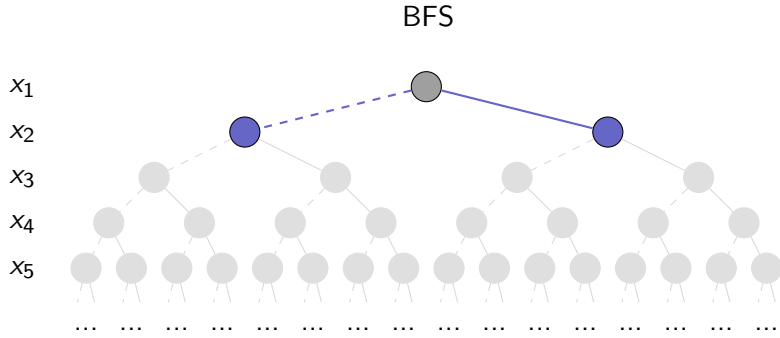
Breadth-first search (BFS) enumerates the search tree level wise.



Starting with the **root node** (empty solution)

Motivation of Beam Search

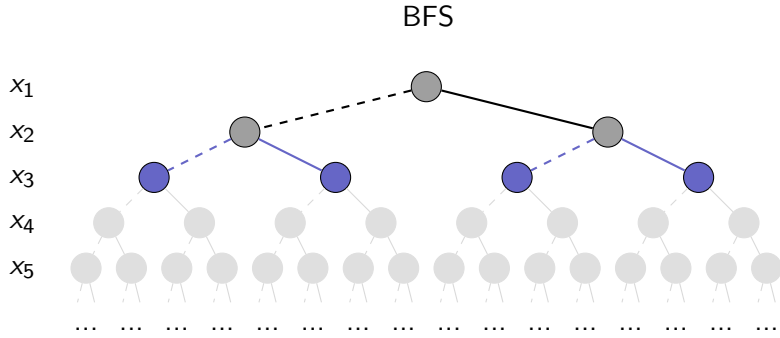
Breadth-first search (BFS) enumerates the search tree level wise.



Assigning 0 or 1 to x_1 (branching on the root node in level 1)

Motivation of Beam Search

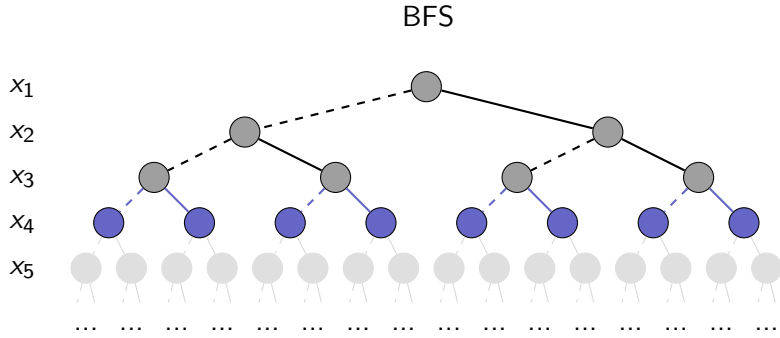
Breadth-first search (BFS) enumerates the search tree level wise.



Assigning 0 or 1 to x_2 (branching on every node in level 2)

Motivation of Beam Search

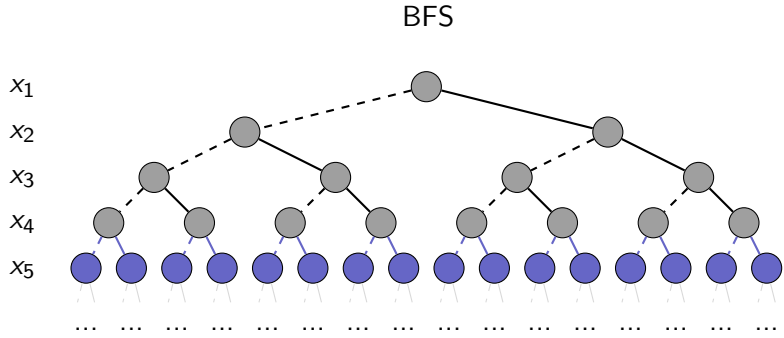
Breadth-first search (BFS) enumerates the search tree level wise.



Assigning 0 or 1 to x_3 (branching on every node in level 3)

Motivation of Beam Search

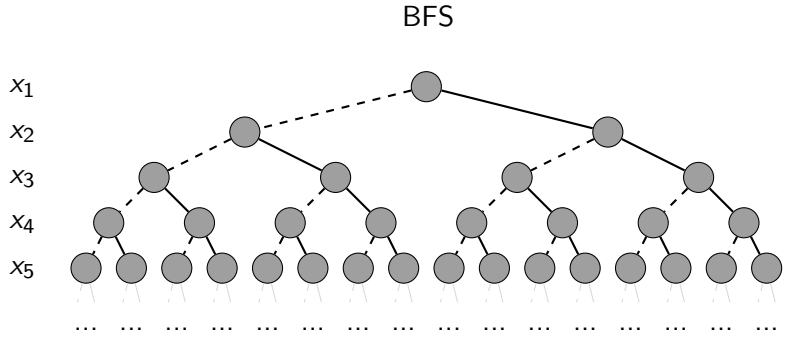
Breadth-first search (BFS) enumerates the search tree level wise.



Assigning 0 or 1 to x_4 (branching on every node in level 4)

Motivation of Beam Search

Breadth-first search (BFS) enumerates the search tree level wise.

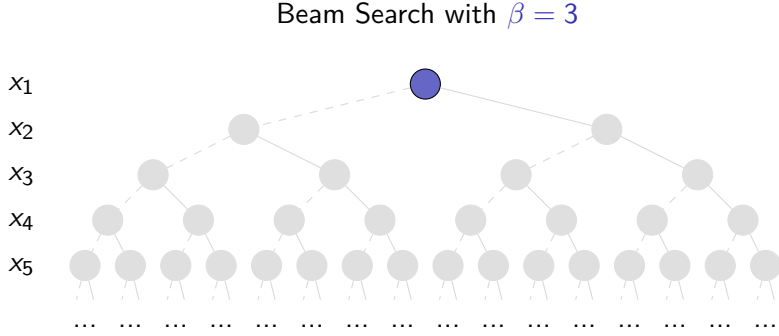


Problem: Space and time complexity: $O(2^n)$

Question: How can we keep the complexity low, more specifically the number of considered nodes in $O(n)$, but still do a meaningful incomplete/heuristic search?

Motivation of Beam Search

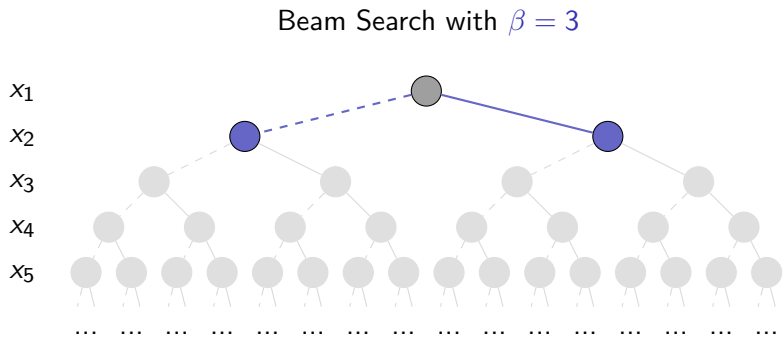
Beam Search only further pursues the β most promising nodes from each level.



Branch on the first node.

Motivation of Beam Search

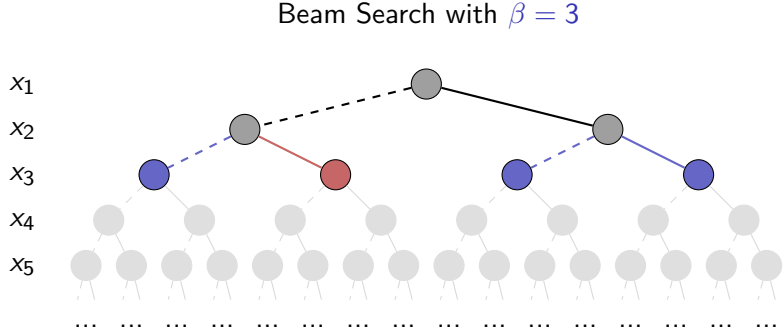
Beam Search only further pursues the β most promising nodes from each level.



Since there are only $2 < 3 = \beta$ nodes to consider, continue with both nodes.

Motivation of Beam Search

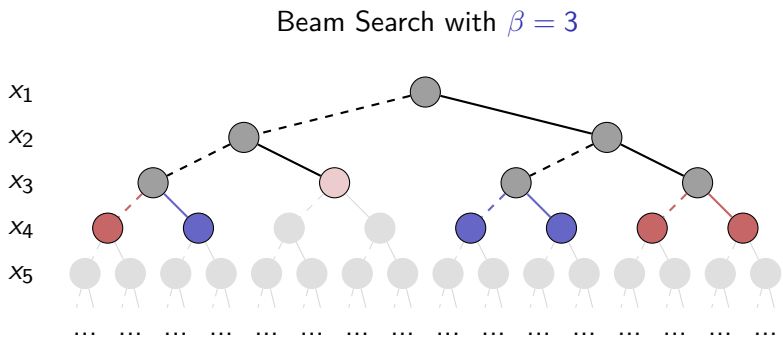
Beam Search only further pursues the β most promising nodes from each level.



Choose the $\beta = 3$ best nodes to continue with and discard the other nodes.

Motivation of Beam Search

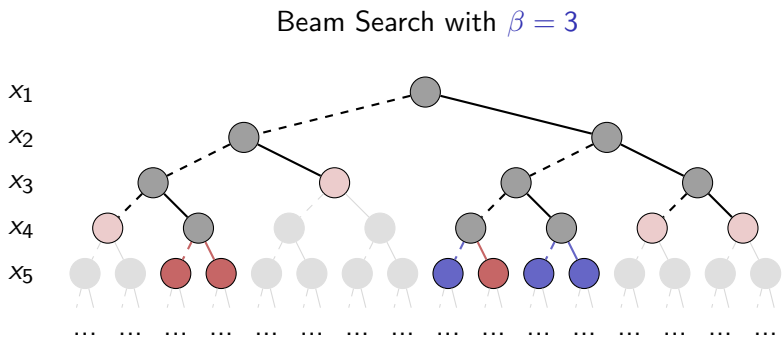
Beam Search only further pursues the β most promising nodes from each level.



Choose the $\beta = 3$ best nodes to continue with and discard the other nodes.

Motivation of Beam Search

Beam Search only further pursues the β most promising nodes from each level.

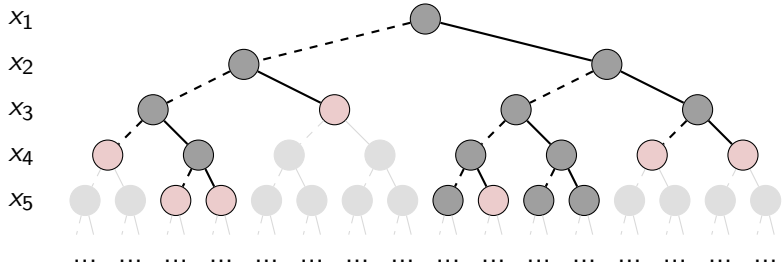


Choose the $\beta = 3$ best nodes to continue with and discard the other nodes.

Motivation of Beam Search

Beam Search only further pursues the β most promising nodes from each level.

Beam Search with $\beta = 3$



Advantages: Only $O(\beta n)$ nodes considered (assuming the number of successors of each node is bounded by a constant).

But: Heuristic approach, i.e., no guarantee for finding an optimum.

Beam Search

Beam Search only further pursues the β most promising nodes from each level.

Question: How do we select the β most promising nodes from each level?

Beam Search

Beam Search only further pursues the β most promising nodes from each level.

Question: How do we select the β most promising nodes from each level?

Let x be a (partial) solution corresponding to a node.
We estimate the objective value of a completion of solution x via

$$f(x) = g(x)[+h(x)]$$

where

- $g(x)$: objective value contribution of partial solution x (costs-so-far)
- $h(x)$: heuristic estimate for further objective value gain when completing x, "value/cost-to-go" (not necessarily a bound, but may be one)

We now only further pursue up to β nodes with the best values $f(x)$.

procedure Beam Search

$x^{\text{best}} = \text{null};$
$S \leftarrow \{\text{empty solution}\};$
while S is not empty **do:**
  $C \leftarrow \emptyset;$
  foreach $x \in S$ **do:**
    if x is complete solution **then**
      if $x^{\text{best}} = \text{null} \vee x$ is better than x^{best} **then**
        $x^{\text{best}} \leftarrow x;$
    else
      $C \leftarrow C \cup$ create all one-step extensions of $x;$
  if $|C| > \beta$ **then**
    $C \leftarrow$ best β elements of C according to $f(\cdot);$
  $S \leftarrow C;$
return $x^{\text{best}};$

Example: Flow Shop Problem

Example: Flow Shop Problem

Given: n jobs $J_1, \ldots, J_n$ and m machines $M_1, \ldots, M_m$

Each job has to be processed sequentially on every machine in the given order of the machines.
A machine can only process one order at a time in an uninterrupted fashion.

$z_{jk} \geq 0$: processing durations for each job j on each machine k

Example: $n = 4$ jobs and $m = 2$ machines; times z_{jk}:

Job	M_1	M_2
J_1	3	1
J_2	2	1
J_3	2	4
J_4	2	3

Goal: Find a feasible schedule that minimizes the sum of finishing times.

Example: Flow Shop Problem

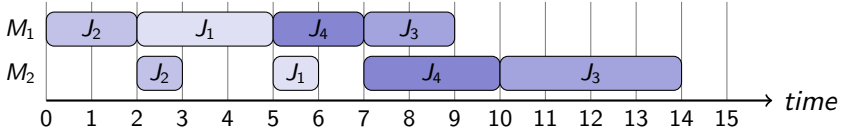
A solution can be represented as a permutation of the jobs $s = \langle \pi_1, \pi_2, \ldots, \pi_n \rangle$.
Jobs are scheduled in the order given by the permutation at the machines as early as possible.

Basic greedy heuristic:

Example: Flow Shop Problem

A solution can be represented as a permutation of the jobs $s = \langle \pi_1, \pi_2, \ldots, \pi_n \rangle$.
Jobs are scheduled in the order given by the permutation at the machines as early as possible.

Basic greedy heuristic:
We always schedule a job next that has the smallest finishing time at the moment.



$f(\langle J_2, J_1, J_4, J_3 \rangle) = 3 + 6 + 10 + 14 = 33$

Let's see, if **Beam Search** performs better!

Example: Flow Shop Problem – Beam Search

For every partial solution s we compute $f(s)$ as defined above:

$$f(s) = g(s) + h(s)$$

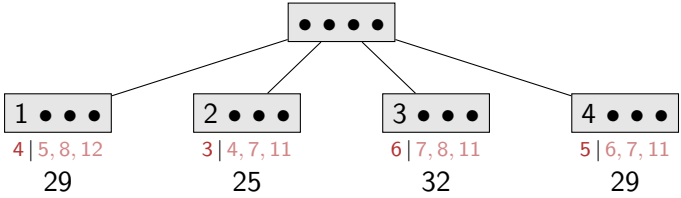
Cost-to-go heuristic $h(x)$:

- Schedule all jobs not assigned yet just on the last machine with their times $z_{.m}$ under the assumption that **no waiting times** occur.

$\rightarrow$ Simple and fast to compute, a lower bound.

We then further pursue those nodes with the β best values of $f(s)$.

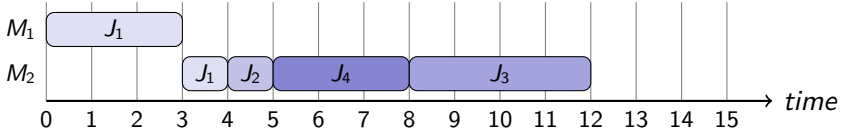
Example: Flow Shop Problem – Beam Search



We start with the empty schedule $\langle \bullet\bullet\bullet\bullet \rangle$.

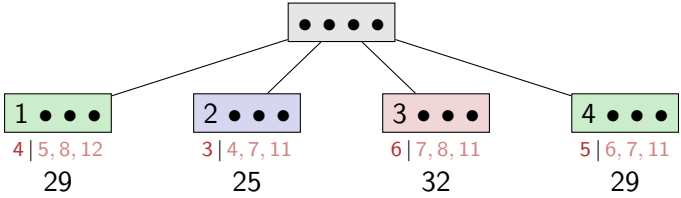
We first branch on the job to schedule first, i.e., π_1 and compute the corresponding estimations for $f(\cdot)$.

Exemplarily shown for $x = \langle 1\bullet\bullet\bullet \rangle$:



$$f(\langle 1\bullet\bullet\bullet \rangle) = g(\langle 1\bullet\bullet\bullet \rangle) + h(\langle 1\bullet\bullet\bullet \rangle) = 4 + 5 + 8 + 12 = 29$$

Example: Flow Shop Problem – Beam Search

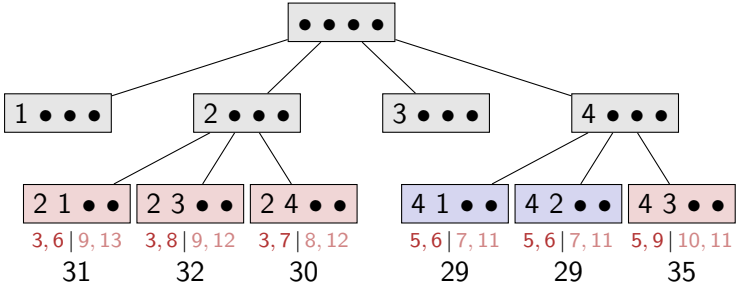


Now we choose the $\beta = 2$ best nodes:

- $\langle 2 \bullet \bullet \bullet \rangle$ is the best node.
- $\langle 1 \bullet \bullet \bullet \rangle$ and $\langle 4 \bullet \bullet \bullet \rangle$ are equally good.
- $\langle 3 \bullet \bullet \bullet \rangle$ is the worst node and is rejected.

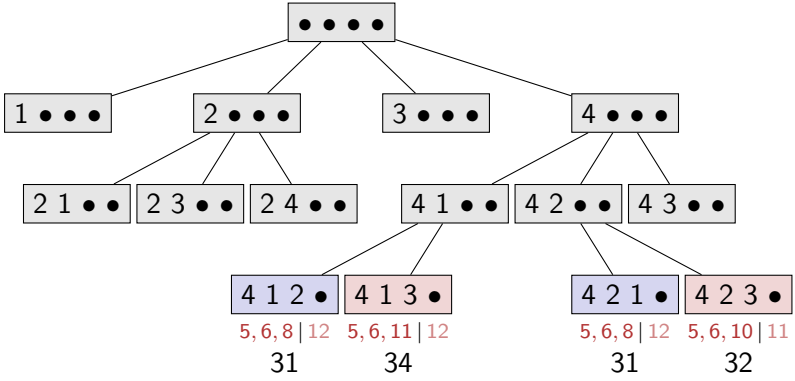
Tie breaking is necessary: We choose $\langle 4 \bullet \bullet \bullet \rangle$ randomly.

Example: Flow Shop Problem – Beam Search

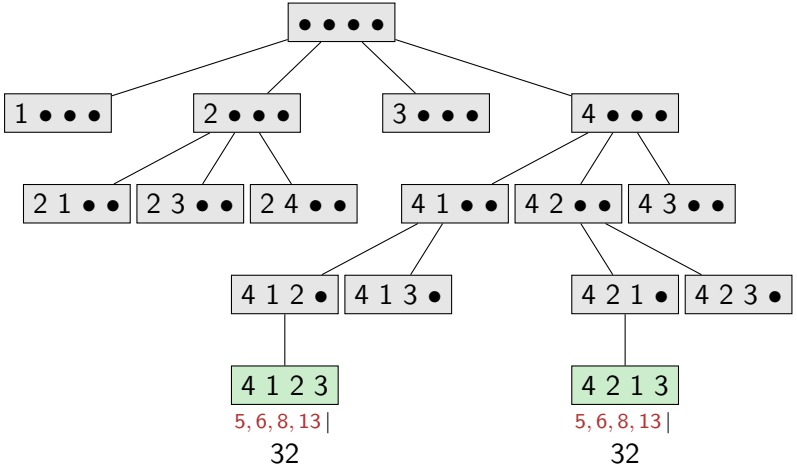


The $\beta = 2$ best nodes are $\langle 41 \bullet\bullet \rangle$ and $\langle 42 \bullet\bullet \rangle$ and are therefore further pursued.

Example: Flow Shop Problem – Beam Search

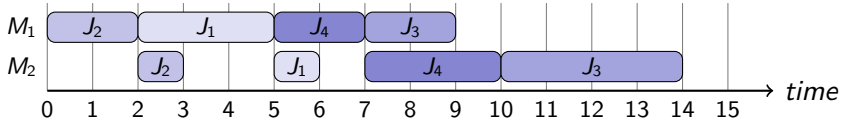


Example: Flow Shop Problem – Beam Search

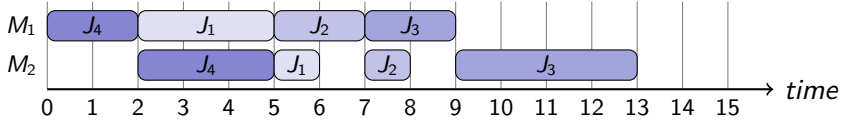


Example Flow Shop: Comparision

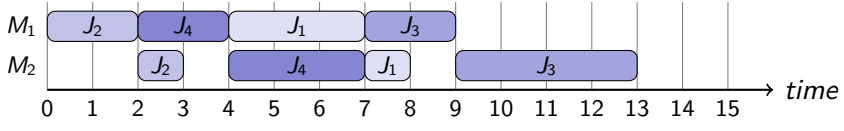
Greedy solution: $f(x) = 3 + 6 + 10 + 14 = 33$



Beam Search solution: $f(x) = 5 + 6 + 8 + 13 = 32$



Optimal solution: $f(x) = 3 + 7 + 8 + 13 = 31$



Beam Search: possible improvements

Question: What can we do to improve the performance of **Beam Search**?

Beam Search: possible improvements

Question: What can we do to improve the performance of **Beam Search**?

Increase the beam-width β

- If we would have taken $\beta = 3$ in above example, we would have found the optimum.
- Note that a Beam Search with $\beta = 1$ is equivalent to a classical greedy construction heuristic.

Improve the selection of variable to branch on next

- Choose a variable that maximizes information gain.
- E.g., MAX-SAT: that variable that occurs in the most still unsatisfied clauses
- E.g., MAX-INDEPENDENT SET: a node with the highest degree

Find a better heuristic function $h(s)$

- E.g., do not just consider the times on the last machine, but also the times on the other machines.
- This, however, is more time-consuming. A good tradeoff between accuracy and time is necessary.

The Pilot Method: Doing Rollouts

The Pilot method

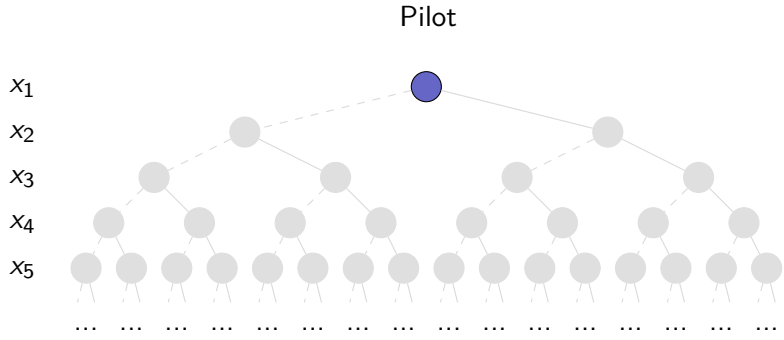
- Basic idea: Try to avoid greedy trap by looking ahead for each possible extension of a partial solution via a rollout before selecting one.
- Principle first suggested by Bertsekas et al. (1997) and called rollout algorithm.
- Further developed and called Pilot method by Duin and Voß (1999).

The Pilot method

- In each major iteration, Pilot evaluates each possible solution extension by applying a complete greedy algorithm.
- The objective value of the greedily constructed solution is used as score of each extension.
- A solution extension with best score is permanently adopted, and the whole process iterated until no further extensions are possible.

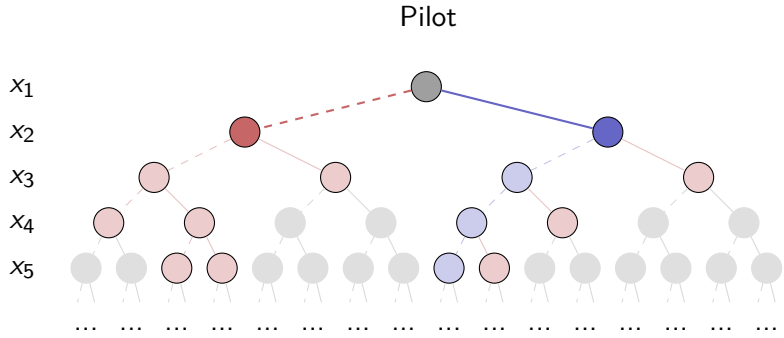
Motivation of Pilot

Pilot evaluates each possible extension by applying after it the whole greedy algorithm.



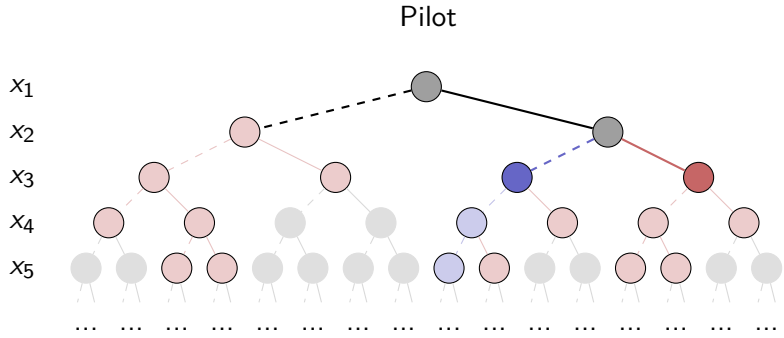
Motivation of Pilot

Pilot evaluates each possible extension by applying after it the whole greedy algorithm.



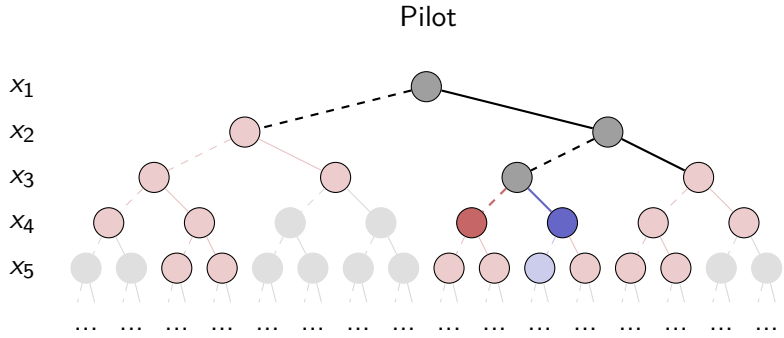
Motivation of Pilot

Pilot evaluates each possible extension by applying after it the whole greedy algorithm.



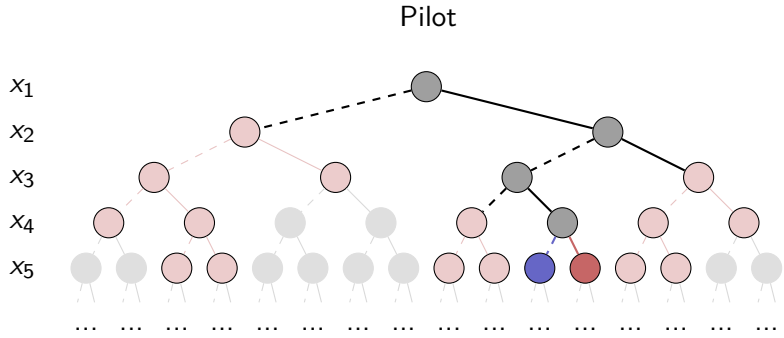
Motivation of Pilot

Pilot evaluates each possible extension by applying after it the whole greedy algorithm.



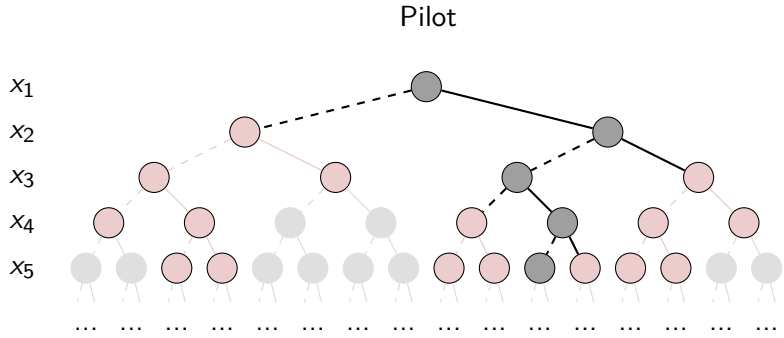
Motivation of Pilot

Pilot evaluates each possible extension by applying after it the whole greedy algorithm.



Motivation of Pilot

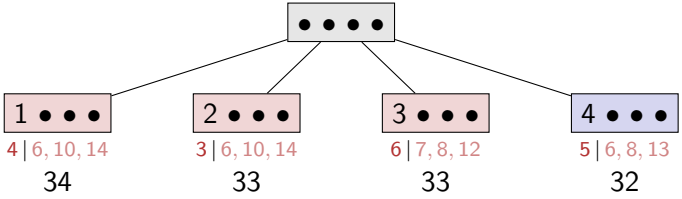
Pilot evaluates each possible extension by applying after it the whole greedy algorithm.



procedure Pilot

$x \leftarrow$ empty solution;
$x^{\text{best}} \leftarrow$ null; // best complete solution
while x is not a complete solution **do:**
  $C \leftarrow$ create all one-step extensions of x;
  $c^{\text{best}} \leftarrow$ null; // best one-step extension
  foreach $c \in C$ **do:**
    if $x^{\text{best}} = \text{null} \vee c$ is not prefix of x^{best} **then**
      $x' \leftarrow$ complete c with greedy algorithm;
    else
      $x' \leftarrow x^{\text{best}}$;
    if $x^{\text{best}} = \text{null} \vee x'$ is better than x^{best} **then**
      $x^{\text{best}} \leftarrow x'$;
      $c^{\text{best}} \leftarrow c$;
  $x \leftarrow c^{\text{best}}$;
return x^{best};

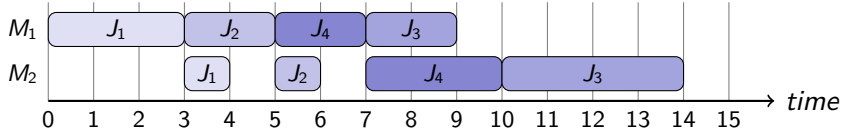
Example: Flow Shop Problem – Pilot method



We start with the empty schedule $\langle \bullet\bullet\bullet\bullet \rangle$.

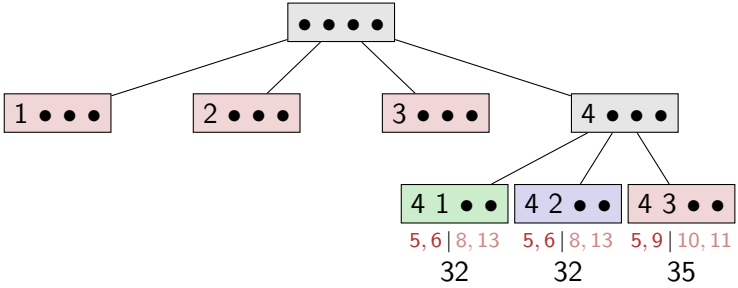
We first branch on π_1, and each partial solution is further extended by applying the greedy algorithm to obtain the score for the extension.

Exemplarily shown for $x = \langle 1\bullet\bullet\bullet \rangle$:



$f(\langle 1\bullet\bullet\bullet \rangle) = g(\langle 1\bullet\bullet\bullet \rangle) + h(\langle 1\bullet\bullet\bullet \rangle) = 4 + 6 + 10 + 14 = 34$

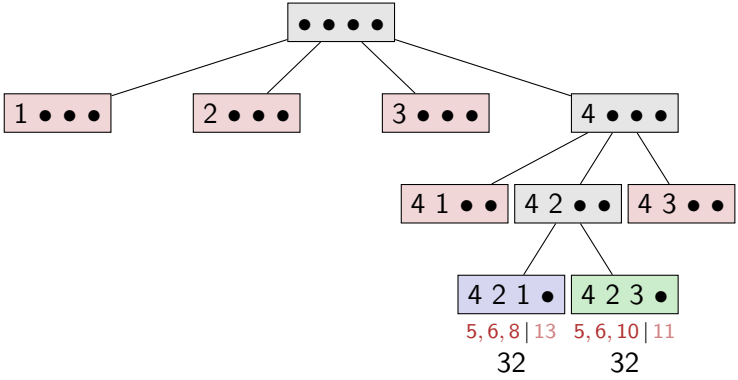
Example: Flow Shop Problem – Pilot method



Tie breaking is necessary since $\langle 41 \bullet \bullet \rangle$ and $\langle 42 \bullet \bullet \rangle$ are equally good.

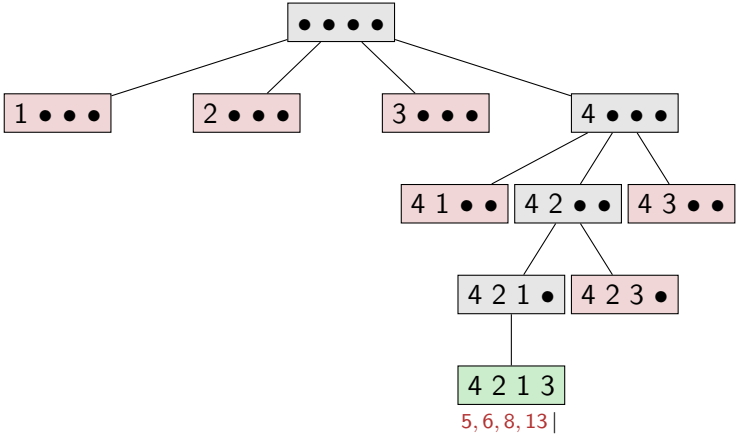
We take $\langle 42 \bullet \bullet \rangle$ randomly.

Example: Flow Shop Problem – Pilot method



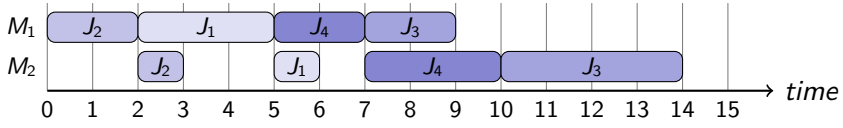
Tie breaking is necessary. We take $\langle 421\bullet \rangle$ randomly.

Example: Flow Shop Problem – Pilot method

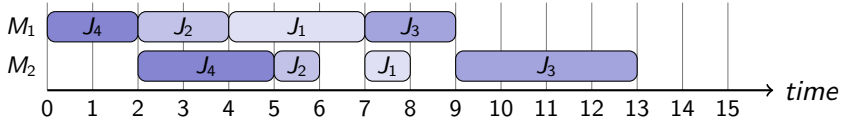


Example: Flow Shop Problem – comparision

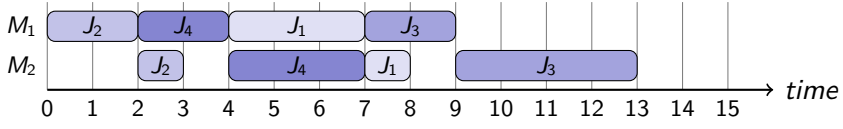
Greedy solution: $f(x) = 3 + 6 + 10 + 14 = 33$



Pilot solution: $f(x) = 5 + 6 + 8 + 13 = 32$



Optimal solution: $f(x) = 3 + 7 + 8 + 13 = 31$



Pilot method: possible speedup

- Pilot expands in general $O(n^2)$ search tree nodes due to the calls of the greedy algorithm for each possible extension in any major iteration (assuming again the number of successors of each node is bounded by a constant).
- This can be reduced to $O(\gamma n)$ nodes, with γ being a parameter, by performing the greedy algorithm always only up to γ steps (rollout length) und using the evaluation of this partial solution as score for the extension.

Conclusions

- Basic construction heuristics often follow a simple scheme, but are in their details highly problem-specific.
- Typically, they are rather fast but yield solutions of only moderate quality for challenging problem instances.
- Beam Search is a natural extension of a one-shot solution construction, we may interpret it as "constructing β solutions simultaneously".
- An alternative is the Pilot method, which evaluates solution extensions more precisely by applying nested greedy rollouts to obtain scores.

Bibliography

D. P. Bertsekas, J. N. Tsitsiklis, and C. Wu. Rollout algorithms for combinatorial optimization. *Journal of Heuristics*, 3(3):245–262, 1997.

G. Clarke and J. W. Wright. Scheduling vehicles from a central delivery depot to a number of delivery points. *Operations Research Quarterly*, 12:568–581, 1964.

C. Duin and S. Voß. The PILOT method: A strategy for heuristic repetition with application to the Steiner problem in graphs. *Networks*, 34(3):181–191, 1999.

B. Lowerre. *The Harpy Speech Recognition System*. PhD thesis, Carnegie Mellon University, 1967.

D. R. Reddy. Speech understanding systems: A summary of results of the five-year research effort. Technical report, Department of Computer Science, Carnegie Mellon University, 1977.